Fraunhofer

Fraunhofer-Gesellschaft

On-Prem AI Inferencing at Scale

Running NVIDIA DGX Workloads with Run:ai and Kubernetes

Umut Tuglu, DevOps Engineer — Fraunhofer-Gesellschaft

Fraunhofer

Fraunhofer-Gesellschaft

Agenda

1 Introduction

2 Motivation

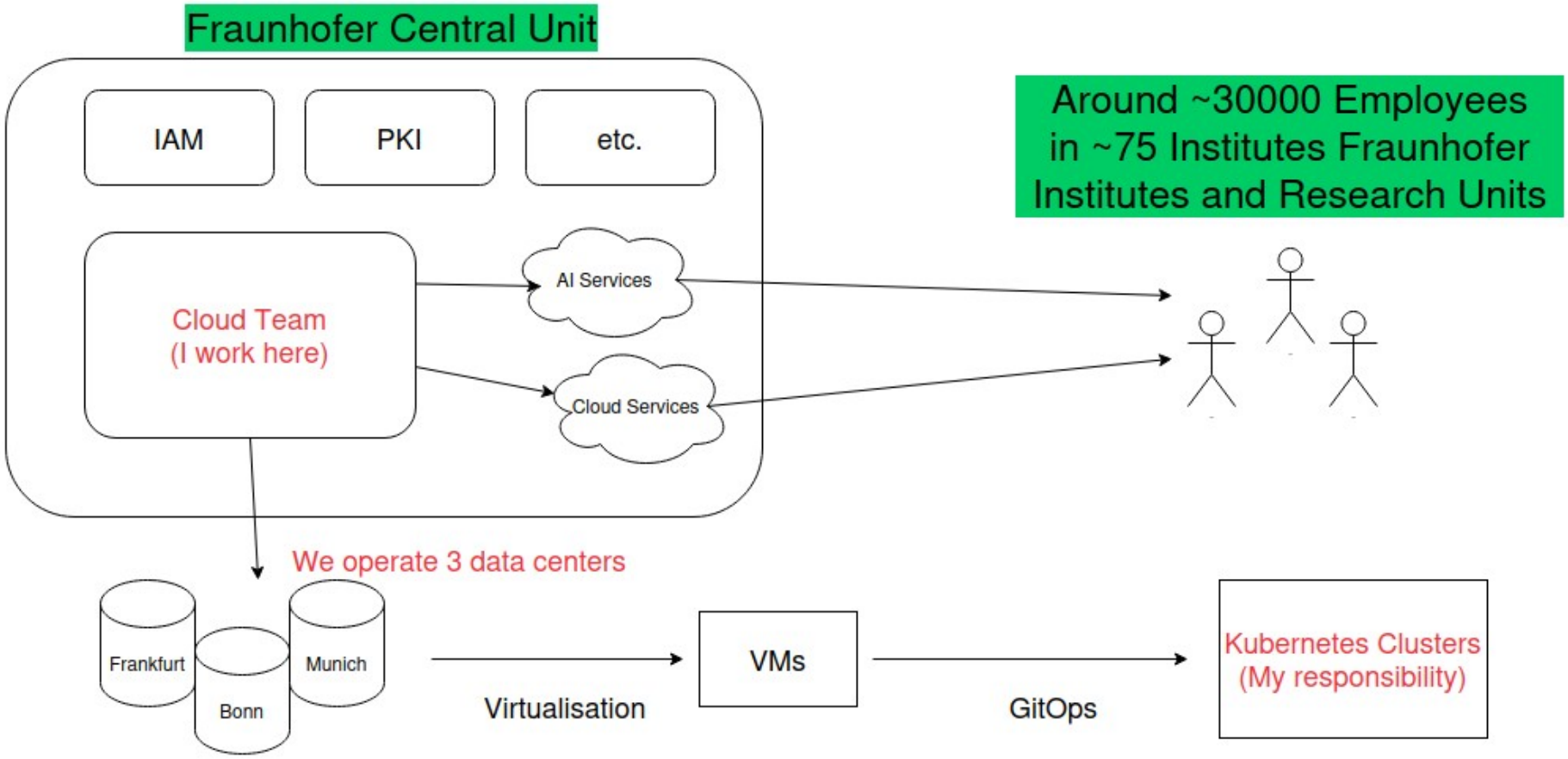
3 Attempt #1: Inferencing with NVIDIA Base Command Manager

4 Attempt #2: Run:ai on our automated Kubernetes Infrastructure

5 In Production across Fraunhofer

Introduction

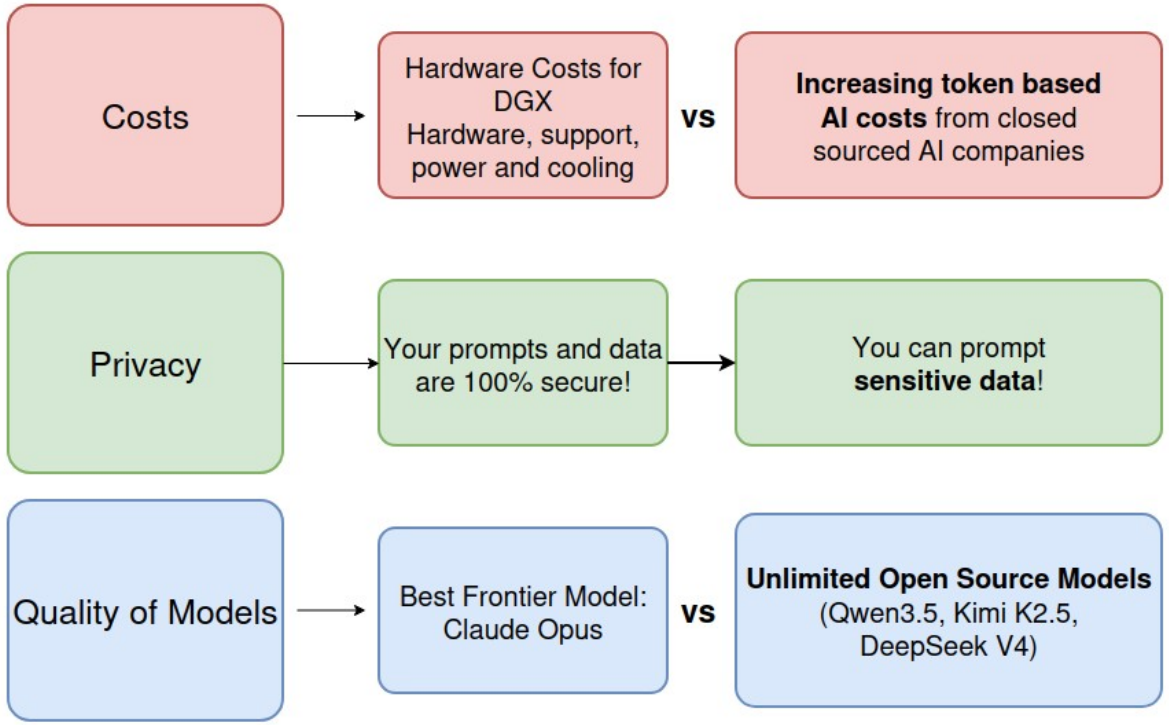
My work at Fraunhofer



© Fraunhofer

Motivation

Why Run Inference On Your Own Infrastructure?



Costs – One Nvidia DGX with 8 H200 GPUs

One Time Hardware Investment and Support -> ~ 250.000 Euros

Maintenance at Equinix (Power, Cooling) -> ~ 60.000 Euros per Year

VS

Skyrocketing Token Costs Across OpenAI, GitHub Copilot and Anthropic

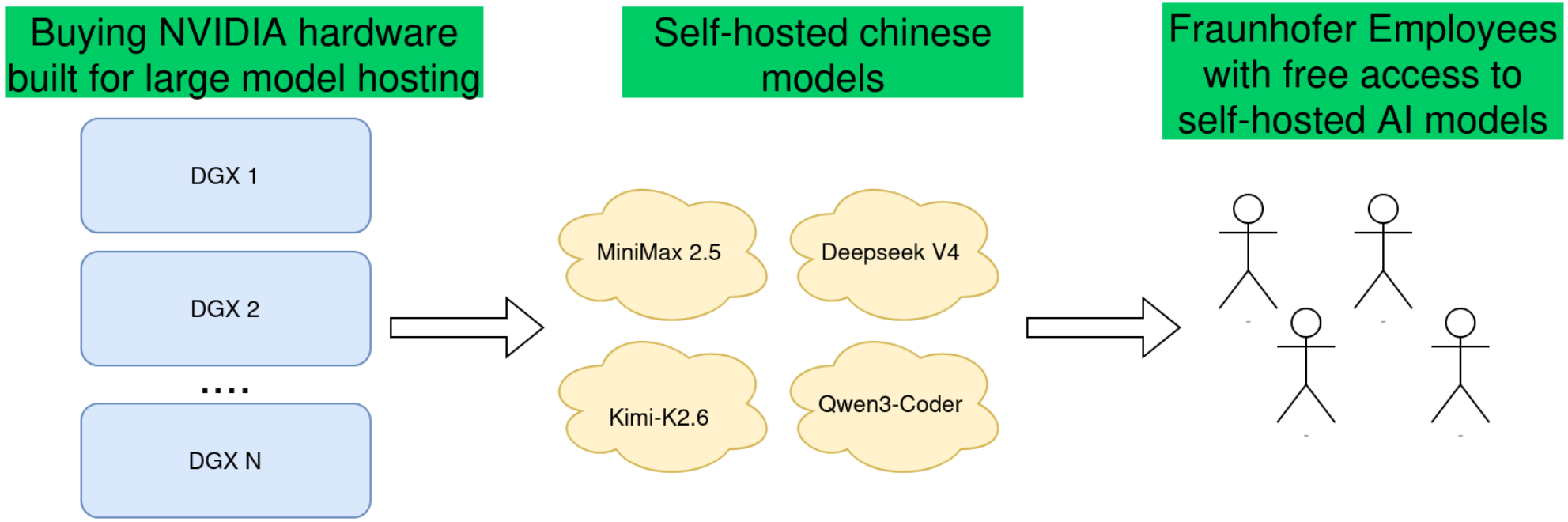
GPT-5.5 increased in price in **May**, **raising** real costs **by up to 92%**

GitHub Copilot token-based billing since June -> costs up to **10x-50x higher** for heavy users

In our own chatbot (Claude, OpenAI) it went up from **40.000€ to 80000€ from March to April this year...**

Motivation

Our Approach: DGX + Self-Hosted Inference



230 Billion Parameter Model (MiniMax 2.5)

~30000 Fraunhofer employees

VRAM as primary bottleneck:

- Which models can be loaded?
- How many requests you can process at the same time?

VRAM needed for 230B Param Model

-> 230 * 1,5 = at least 345 GB needed

At least a half a DGX (4xH200 GPUs) needed

Motivation

What is an NVIDIA DGX?

A DGX is a **dedicated AI compute server**.

-> Designed for both model training and **large-scale inference workloads**.

8× NVIDIA H200

One server and eight GPUs

141 GB VRAM per GPU

GPU memory = what limits which AI models you can run and how many requests you can process at the same time. More = bigger models

GPUs talk directly to each other

Via NVLink: Not through the CPU. Makes multi-GPU inference fast

Bare metal -> No VMs

Direct hardware access for maximum performance



© Fraunhofer

Fraunhofer

Fraunhofer-Gesellschaft

On-Prem AI Inferencing

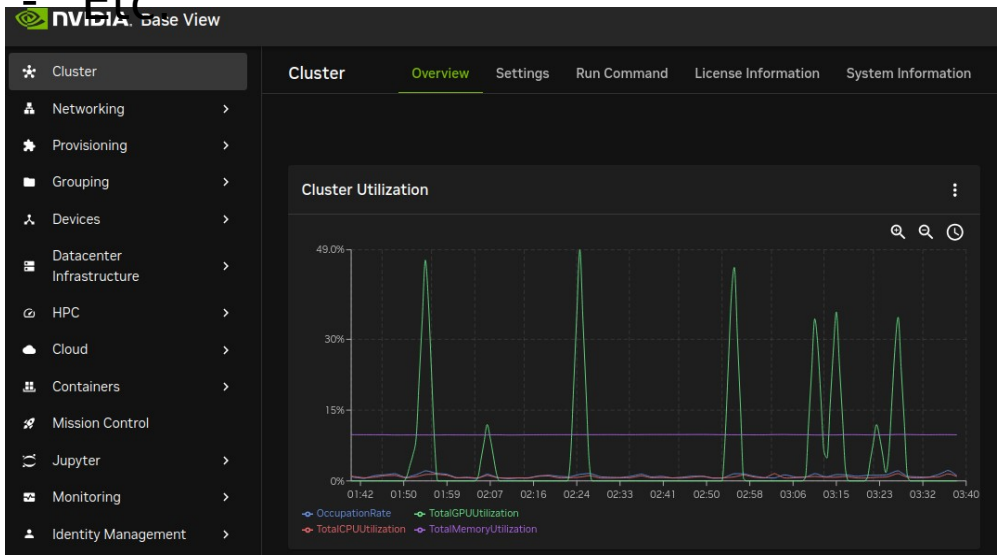
Attempt #1 — Inferencing with NVIDIA Base Command Manager

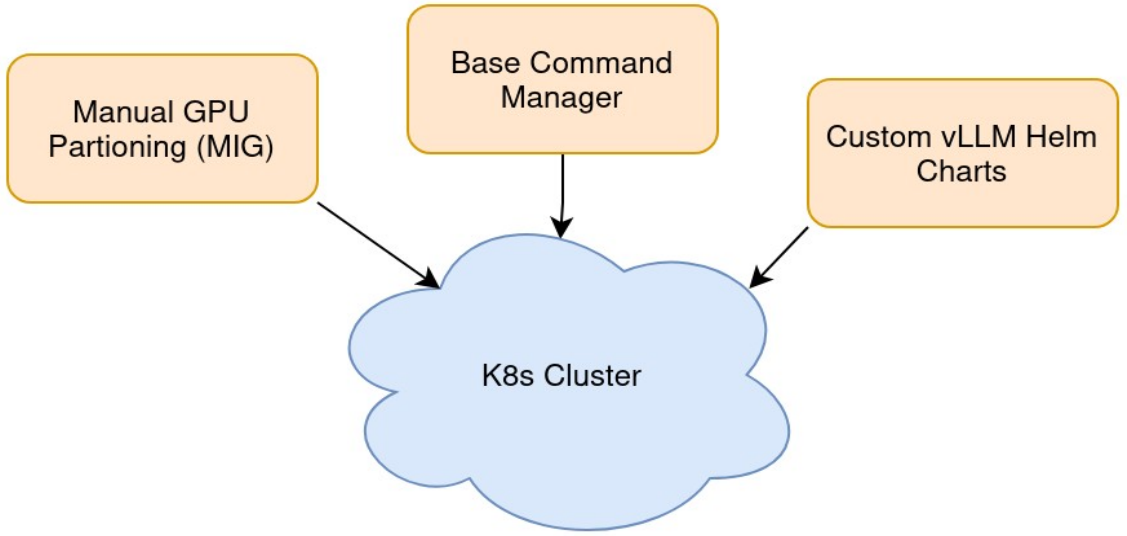
Base Command Manager Setup

What is the Base Command Manager?

Included with every DGX: **Base Command Manager (BCM) is NVIDIA software that automates the full lifecycle of AI cluster management from provisioning to monitoring.**

- Ubuntu Image with drivers
- Creating a K8s cluster via TUI
- Etc.





© Fraunhofer

Base Command Manager Setup

Problems & Goals

What were the problems?

- A lot of CLI-TUI-only config -> no GitOps
- BCM overwrote custom nginx config and reboots wiped all network routes
  - A lot of Custom Bash Scripts to fix the buggy environment
- A lot of custom work -> manual GPU Partitioning, manual vLLM Helm charts
  - Difficult to implement Time Slicing
- No Billing for our "GPU as a Service"
- Adding nodes was painful
- Cluster upgrades only via kubeadm were painful

Where do we want to go?

- GPU complexity abstracted away by Run:ai
- Technical overhead abstracted away by Run:ai
- DGX nodes easily onboarded via Rancher automation
- Cluster lifecycle automated via Rancher
- Full GitOps -> Run:ai deployed via Helm charts and ArgoCD

© Fraunhofer

Fraunhofer

Fraunhofer-Gesellschaft

On-Prem AI Inferencing

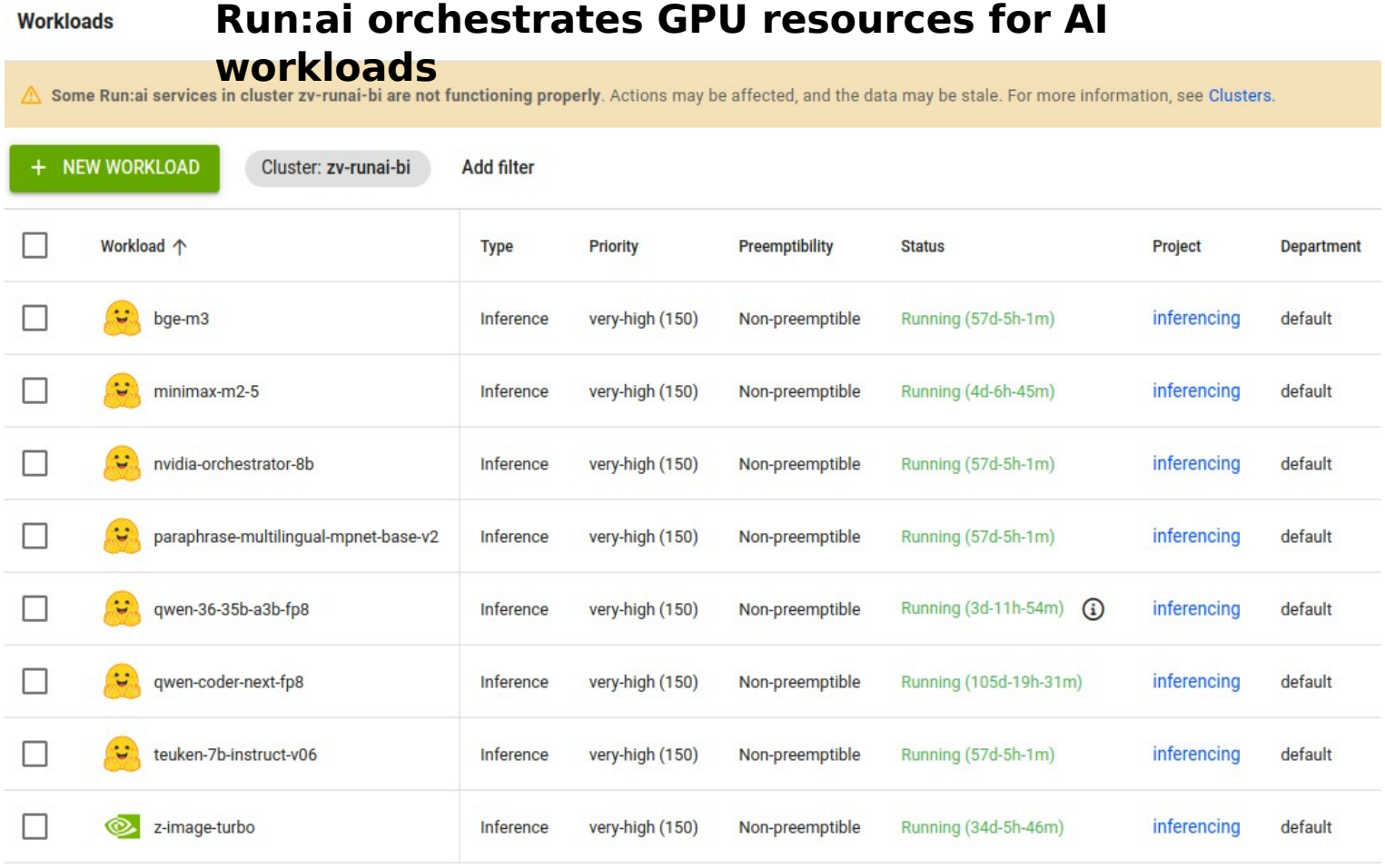
Attempt #2 — Run:ai on our automated Kubernetes Infra

Run:ai

Overview

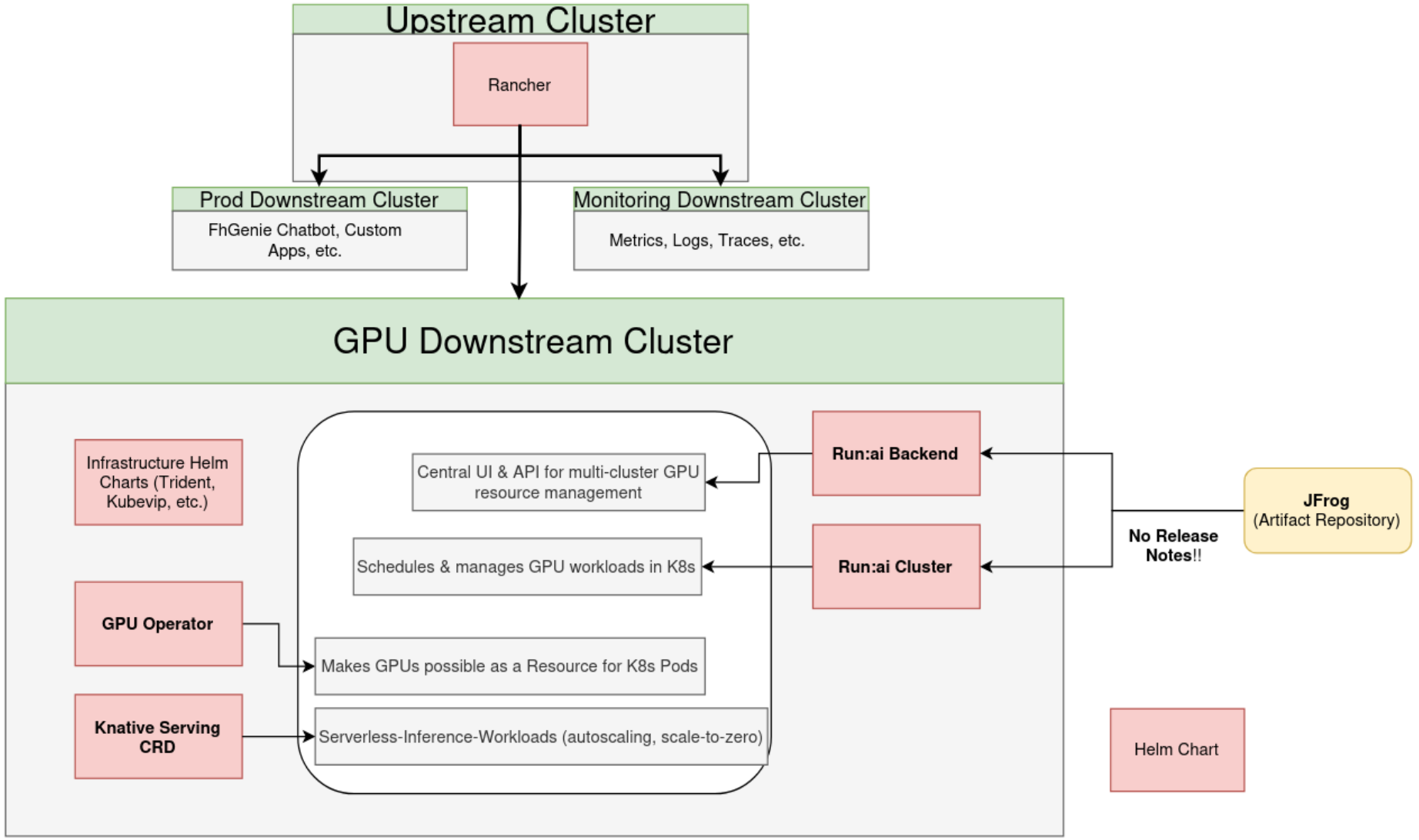
- **Reduced Technical Overhead**
  - Without Run:ai: Implement technical stack from scratch
  - With Run:ai: MIG & Time-Slicing out of the box – GPUs as native K8s resources
- **GitOps-Driven Automation**
  - Without Run:ai: GPU stack managed manually across tools
  - With Run:ai: Entire stack (GPU Operator, Knative, control-plane) reconciled via ArgoCD
- **Billing**
  - Without Run:ai: We have to build it ourselves
  - With Run:ai: Built-in billing & GPU usage tracking per team/project
- **License Costs**
  - **5 Years and 16 GPUs -> 50.000€**

Run:ai orchestrates GPU resources for AI workloads



Rancher Setup

K8s Infra

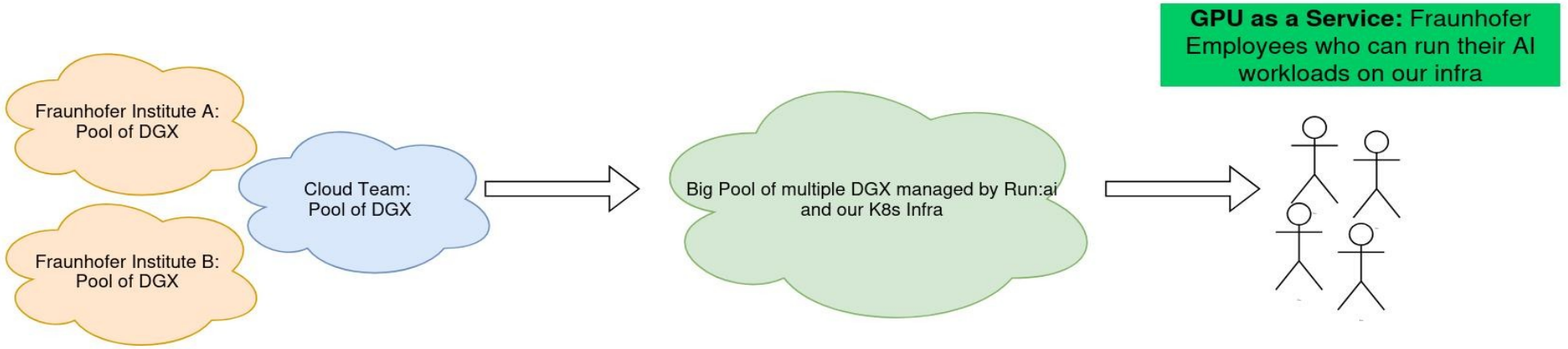


Scaling our DGX infra

Adding new DGX

NAME↑	STATUS	ROLE
dgxnode000	Ready	worker
dgxnode001	Ready	worker
k8s-gpu-master-7hptc-g8xqg	Ready	control-plane,etcd
k8s-gpu-master-7hptc-q4nbl	Ready	control-plane,etcd
k8s-gpu-master-7hptc-zp6xs	Ready	control-plane,etcd
k8s-gpu-worker-k9z8b-826ng	Ready	worker
k8s-gpu-worker-k9z8b-mkrns	Ready	worker
k8s-gpu-worker-k9z8b-wpg5g	Ready	worker

Scaling the infra by attaching a DGX as a new node to the cluster



© Fraunhofer

Cutting Edge Technology

Nvidia Hardware & Software Challenges

Temperature of one GPU fluctuating between 70°C-90°C

solution

Support came and replaced the GPU in 1-2 weeks

Exposing metrics via Prometheus and monitoring temperature



Run:ai Control Plane Helm Chart breaking changes and bugs (over 10 tickets)

e.g. new PostgreSQL 17 database was not compatible with Run:ai init containers (although advertised in their docs)

Downscaling the PostgreSQL version from 17 to 16 was not possible. So, we opened a **ticket with NVIDIA**

After 1-2 weeks they sent us a **patch via a bash script**

© Fraunhofer

Fraunhofer

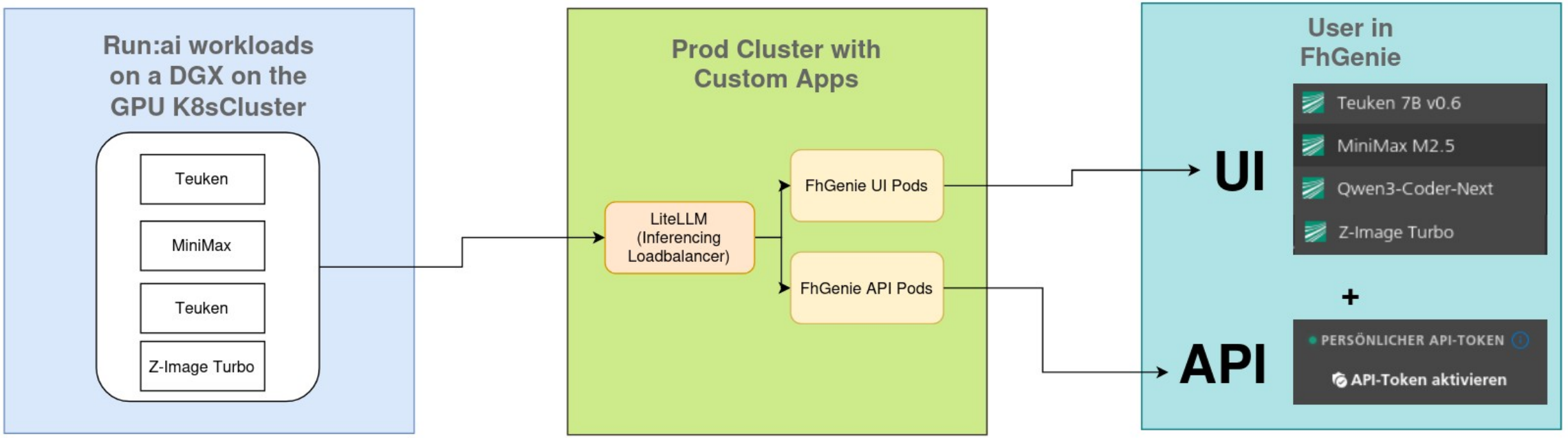
Fraunhofer-Gesellschaft

On-Prem AI Inferencing

In production across Fraunhofer

Inferencing, end to end

AI Services

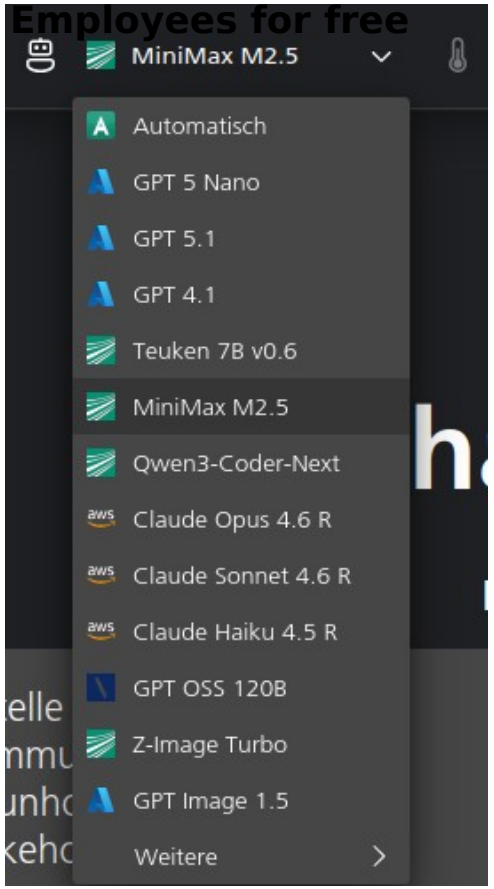


© Fraunhofer

What Runs on It Today

From chatbot to observability

Chatbot UI + API for Fraunhofer Employees for free



Grafana LLM Assistant



OpenCode with self-hosted



© Fraunhofer

Key Takeaways

- **Tradeoff: Costs and Privacy vs Quality of Models**
- **Managing the K8s cluster and Inferencing via GitOps and Automation**
- **Infrastructure with maximum flexibility where you can scale up by adding DGX Nodes**
- **Reduced Operational Overhead with Run:ai**
- **Connect self-hosted AI models to any compatible tool where it adds real value**

Fraunhofer

Fraunhofer-Gesellschaft

On-Prem AI Inferencing at Scale

Thank you — Questions?

Umut Tuglu · Fraunhofer-Gesellschaft